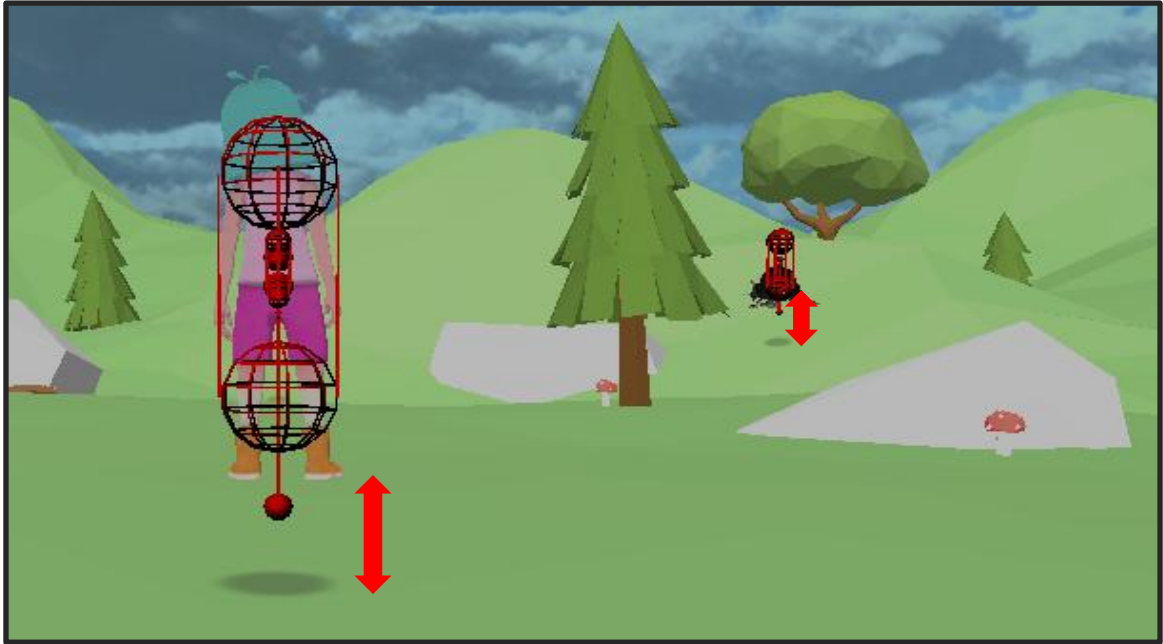


配置の簡単なデバッグ機能

3Dステージ上にプレイヤーやエネミーを配置する場合、座標をどの値にすれば良いか、困ることがあります。



現状でも、プレイヤーとエネミーのY座標が地面に接地していないため、フェード中は、それぞれ浮いている状態です。

プレイヤー座標を画面に描画しつつ、プレイヤーを歩かせて、座標を確認するという方法でも良いですが、オブジェクトの数が多くなると手間になります。

手間を省くため、ゲーム本編機能で使えるような配置の技術を使って、デバッグ機能を作成していきたいと思います。

```
SceneManager.h
```

```
public:
```

```
    ~ 省略 ~
```

```
// シーン管理用
```

```
enum class SCENE_ID
```

```
{
```

```
    NONE,
```

```
    TITLE,
```

```
    GAME,  
    DEBUG,  
};
```

SceneManager.cpp

```
#include "../Scene/DebugScene.h"
```

```
void SceneManager::Init(void)  
{
```

```
    ~ 省略 ~
```

```
    // 初期シーンの設定
```

```
    DoChangeScene(SCENE_ID::DEBUG);
```

```
}
```

```
void SceneManager::DoChangeScene(SCENE_ID sceneId)  
{
```

```
    ~ 省略 ~
```

```
    switch (sceneId_)
```

```
    {
```

```
    case SCENE_ID::TITLE:
```

```
        scene_ = new TitleScene();
```

```
        break;
```

```
    case SCENE_ID::GAME:
```

```
        scene_ = new GameScene();
```

```
        break;
```

```
    case SCENE_ID::DEBUG:
```

```
        scene_ = new DebugScene();
```

```
        break;
```

```
    }
```

```
    ~ 省略 ~
```

```
}
```

```
DebugScene.h
```

```
#pragma once
```

```
#include <vector>
```

```
#include <DxLib.h>
```

```
#include "SceneBase.h"
```

```
class Stage;
```

```
class DebugScene : public SceneBase  
{
```

```
public:
```

```
    // コンストラクタ
```

```
    DebugScene(void);
```

```
    // デストラクタ
```

```
    ~DebugScene(void) override;
```

```
    // 初期化
```

```
    void Init(void) override;
```

```
    // 更新
```

```
    void Update(void) override;
```

```
    // 描画
```

```
    void Draw(void) override;
```

```
    // 解放
```

```
    void Release(void) override;
```

```
private:
```

```
    // ステージ
```

```
    Stage* stage_;
```

```
    // デバッグポイント群
```

```
    std::vector<VECTOR> points_;
```

```
    // デバッグポイントの配置
```

```

void PlaceDebugPoint(void);

// デバッグポイントの保存
void SavePoints(void);

};

```

DebugScene.cpp

```

#include <fstream>
#include <DxLib.h>
#include "../Common/Vector2.h"
#include "../Manager/InputManager.h"
#include "../Manager/SceneManager.h"
#include "../Manager/Camera.h"
#include "../Object/Actor/Stage.h"
#include "../Object/Collider/ColliderModel.h"
#include "DebugScene.h"

```

```

DebugScene::DebugScene(void)
:
    SceneBase(),
    stage_(nullptr)
{
}

```

```

DebugScene::~~DebugScene(void)
{
}

```

```

void DebugScene::Init(void)
{

```

```

    // ステージ生成
    stage_ = new Stage();
    stage_>Init();

```

```

    // カメラの追従設定
    Camera* camera = sceMng_.GetCamera();
    camera->ChangeMode(Camera::MODE::FREE);

```

```

}

void DebugScene::Update(void)
{

    // ステージ更新
    stage_>Update();

    // デバッグポイントの配置
    PlaceDebugPoint();

}

void DebugScene::Draw(void)
{

    // ステージ描画
    stage_>Draw();

    // デバッグポイント群を球体描画
    int y = 20;
    for (const auto& point : points_)
    {

        DrawSphere3D(
            point,
            30.0f,
            16,
            GetColor(255, 0, 0),
            GetColor(255, 0, 0),
            false);

        DrawFormatString(20, y,
            0x000000, "座標(%.2f, %.2f, %.2f)",
            point.x, point.y, point.z);

        y += 20;

    }

}

```

```

void DebugScene::Release(void)
{

    // ステージ解放
    stage_>Release();
    delete stage_;

    // デバッグポイント群
    points_.clear();

}

void DebugScene::PlaceDebugPoint(void)
{

    const auto& ins = InputManager::GetInstance();

    // クリックした場所にデバッグポイントを配置
    if (ins.IsTrgMouseLeft())
    {

        // マウス座標の取得
        Vector2 mousePos = ins.GetMousePos();

        // スクリーン座標をVECTOR構造体に変換
        VECTOR screenPos = VECTOR();
        screenPos.x = static_cast<float>(mousePos.x);
        screenPos.y = static_cast<float>(mousePos.y);
        // zが1.0fでカメラの最奥になる
        screenPos.z = 1.0f;

        // マウスの2D座標から3D座標へ変換
        VECTOR worldPos = ConvScreenPosToWorldPos(screenPos);

        // ステージのモデルコライダを取得
        const ColliderBase* collider = stage_>GetOwnCollider(
            static_cast<int>(Stage::COLLIDER_TYPE::MODEL));

        if (collider == nullptr)
        {

```

```

        return;
    }
    const ColliderModel* colliderModel =
        dynamic_cast<const ColliderModel*>(collider);

    // カメラ情報を取得
    const auto& camera = SceneManager::GetInstance().GetCamera();

    // カメラの位置からカメラ最奥のワールド座標へ向けてレイを飛ばす
    auto hit = MVICollCheck_Line(
        colliderModel->GetFollow()->modelId, -1,
        camera->GetPos(),
        worldPos
    );

    if (hit.HitFlag)
    {
        // 衝突地点をデバッグポイント群に追加
        points_.push_back(hit.HitPosition);
    }

}

// 右クリックで最後のデバッグポイントを削除
if (ins.IsTrgMouseRight())
{
    if (points_.size() > 0)
    {
        points_.pop_back();
    }
}

if (ins.IsTrgDown(KEY_INPUT_SPACE))
{
    // デバッグポイントの保存
    SavePoints();
}

}

```

```

void DebugScene::SavePoints(void)
{

    std::ofstream ofs("Data/Csv/PointSave.txt");
    if (!ofs) {
        return;
    }

    // 形式: x y z
    for (const VECTOR& p : points_) {
        ofs << p.x << " " << p.y << " " << p.z << "\n";
    }

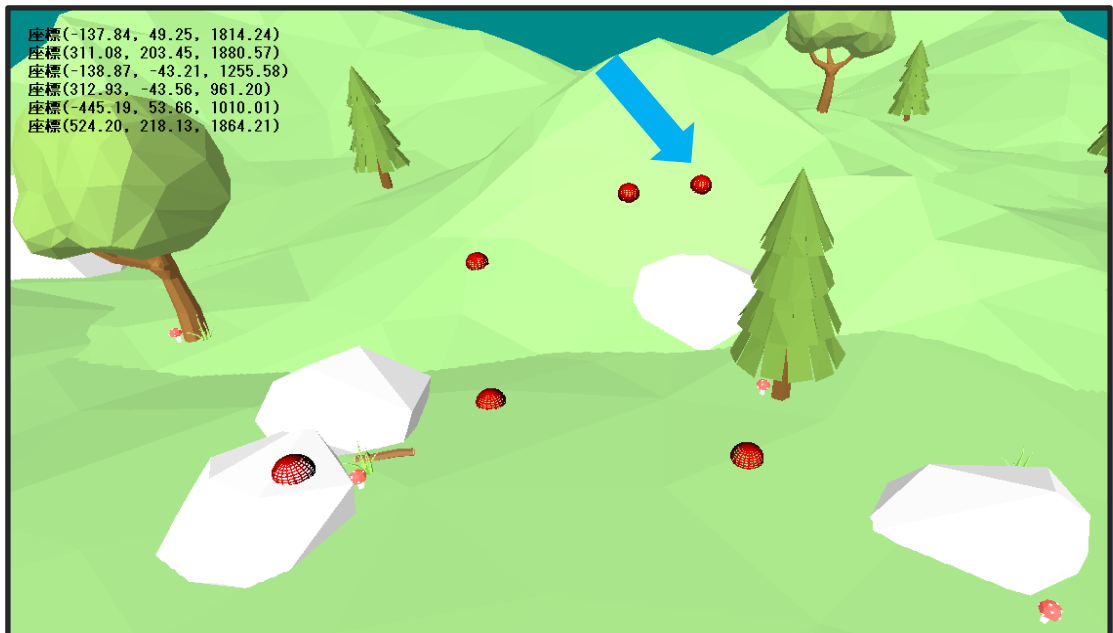
    ofs.close();

}

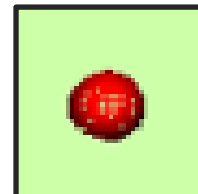
```


解説

- DebugSceneでは、カメラをフリーモードに設定しているので、ステージ内を好きに移動できる
- マウスを左クリックすると、マウス座標(2D)から、ワールド座標(3D)に変換され、3D上のステージに球体が描画される
- 複数回、左クリックするとその分、球体が増え、右クリックすると、最後に増えた球体が削除される
- SPACEボタン押下で、外部ファイルに出力

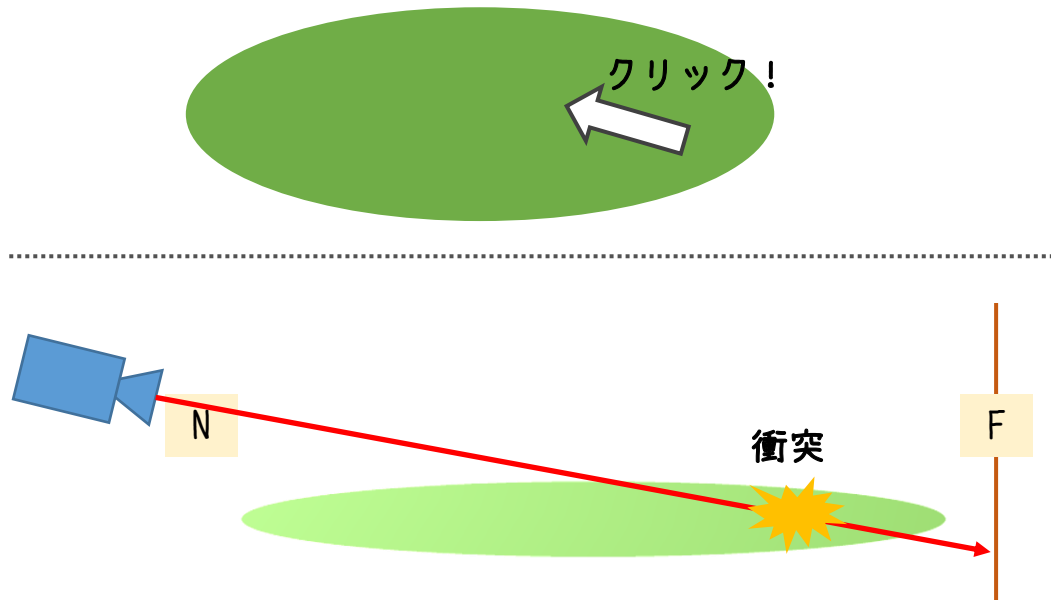


座標(-137.84, 49.25, 1814.24)
座標(311.08, 203.45, 1880.57)
座標(-138.87, -43.21, 1255.58)
座標(312.93, -43.56, 961.20)
座標(-445.19, 53.66, 1010.01)
座標(524.20, 218.13, 1864.21)



- DxDLibのConvScreenPosToWorldPos関数は、マウス座標(2D)をワールド座標(3D)に変換することができる

この時、引数に渡すZ値は、0.0~1.0に正規化されたカメラのNEAR、FARになります。



- カメラ位置から、マウス座標(2D)をワールド座標(3D)に変換した、クリック座標を線分で繋ぎ、ステージモデルと衝突した座標が3D空間上の配置座標になります
- 外部出力結果

```

ファイル(F) 編集(E) 変換(C) 検索(S) ツール(T) 設定(O) ウィンドウ(W) ヘルプ(H)
[Icons]
0 1. 2. 3. 4.
1 -57.6951 -38.7925 499.305<
2 240.682 -41.9895 315.62<
3 153.453 -38.7051 125.385<
4 353.859 -10.485 1402.11<
5 270.933 166.135 1846.35<
6 143.06 165.204 1910.9<
7 -241.135 -35.0819 129.108<
[EOF]

```